

Preguntas de Comprensión

Serie A: Arquitectura MVVM

1. ¿Por qué el acceso directo de los ViewModels a ProductRetrofitClient.apiService viola el patrón MVVM y qué capa intermedia debería existir?

Porque el ViewModel solo debería manejar el estado de la pantalla, no encargarse de obtener los datos de internet. Al llamar directamente a la API queda muy ligado a Retrofit, es más difícil de probar y cualquier cambio en la red obliga a modificarlo. Entre ambos debe existir un **Repositorio** (por ejemplo, ProductRepository), que es el que se comunica con la API y le entrega los datos al ViewModel.

2. ¿Cuál es la responsabilidad de un StateFlow en el ViewModel y por qué se expone como StateFlow y no como MutableStateFlow?

El StateFlow guarda el **estado actual de la pantalla** (cargando, datos, error, etc.) y avisa a la UI cada vez que cambia para que se actualice. Se expone como StateFlow (solo lectura) para que la pantalla únicamente pueda observar el estado y no modificarlo; solo el ViewModel puede cambiarlo, así hay una sola fuente de verdad.

3. ¿Qué función cumple LaunchedEffect en LoginScreen y qué sucedería si se elimina la key state.loginExit?

LaunchedEffect sirve para ejecutar una acción, en este caso **navegar a la lista de productos**, solo cuando cambia el valor de state.loginExit, y no cada vez que se redibuja la pantalla. Si se quita esa key, el efecto se ejecutaría solo una vez al abrir la pantalla (cuando el login aún no es exitoso) y, al iniciar sesión correctamente, **la app no navegaría**.

Serie B: Navegación y Retrofit

1. Explique el flujo completo del valor productId desde que el usuario toca un producto hasta que se muestra en ProductDetailScreen.

Cuando el usuario toca un producto en la lista, se toma su id y se navega a la ruta `product_detail/{productId}`, reemplazando `{productId}` por el valor real (por ejemplo, `product_detail/5`). El NavHost recibe esa ruta, extrae el productId y se lo pasa a ProductDetailScreen. La pantalla se lo envía a su ViewModel, que consulta el producto a la API y actualiza el estado, y finalmente la pantalla **muestra el detalle del producto**.

2. ¿Qué hace `popUpTo(AppRoutes.LOGIN) { inclusive = true }` al iniciar sesión y qué pasaría si no se incluyera?

Elimina la pantalla de login del historial de navegación al entrar a la lista de productos. Si no se incluyera, al presionar el botón **Atrás** el usuario volvería al login aunque ya haya iniciado sesión.

3. ¿Cuál es el propósito del `HttpLoggingInterceptor` con nivel `BODY` en `ProductRetrofitClient` y dónde se observan sus resultados?

Sirve para **revisar la comunicación con la API**: muestra cada petición y respuesta, y con el nivel `BODY` incluye también los datos completos que se envían y se reciben. Es útil para encontrar errores. Los resultados se ven en la ventana **Logcat** de Android Studio mientras la app se ejecuta en el emulador o en el celular.